

Apostila 4 — Econometria com Python

1. Introdução

Você vai usar statsmodels para fazer a parte econométrica. É a biblioteca mais completa.

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.tsa.stattools import adfuller
from bcb import sgs
```

2. Regressão Linear Simples

2.1. Primeiro modelo: consumo vs renda

```
# Dados simulados (Consumo Keynesiano)
np.random.seed(42)
renda = np.arange(1000, 11000, 1000)
consumo = 200 + 0.8 * renda + np.random.normal(0, 100, len(renda))

df = pd.DataFrame({"renda": renda, "consumo": consumo})

# Adicionar constante (beta_0)
X = sm.add_constant(df["renda"])
y = df["consumo"]

# Estimar
modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

2.2. Interpretando o summary

```
OLS Regression Results

=====
Dep. Variable:          consumo    R-squared:                0.996
Adj. R-squared:          0.995
F-statistic:            1782      Prob (F-statistic):        2.58e-10
=====
               coef    std err          t      P>|t|
-----
const         226.7169    67.233      3.372    0.010
renda          0.7959     0.019    42.214    0.000
=====
```

Item	O que significa
const (226.7)	Consumo autônomo (se renda = 0)
renda (0.796)	Propensão marginal a consumir
R-squared (0.996)	99,6% da variação do consumo é explicada pela renda
P> t (< 0.05)	Coefficiente estatisticamente significativo
std err	Erro padrão do coeficiente

2.3. Extrair resultados

```
modelo.params      # coeficientes
modelo.pvalues     # p-valores
```

```
modelo.rsquared      # R²
modelo.resid         # resíduos
modelo.fittedvalues  # valores preditos
modelo.conf_int()    # intervalos de confiança (95%)
modelo.bse           # erros padrão
```

3. Regressão Múltipla

```
# Vários determinantes do consumo
df["poupanca"] = df["renda"] - df["consumo"]
df["juros"] = 5 + np.random.normal(0, 1, len(df))

X = sm.add_constant(df[["renda", "juros", "poupanca"]])
y = df["consumo"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

4. Curva de Phillips (dados reais)

Relação entre inflação e desemprego.

```
from bcb import sgs

# IPCA (433) e Desemprego (24369 - taxa de desocupação)
ipca = sgs.get({"IPCA": 433}, start="2012-01-01")
desemprego = sgs.get({"DESEMPREGO": 24369}, start="2012-01-01")

df = pd.merge(ipca, desemprego, on="Date")
df.columns = ["ipca", "desemprego"]
df["ipca_lag1"] = df["ipca"].shift(1) # inflação passada

df = df.dropna()

X = sm.add_constant(df[["desemprego", "ipca_lag1"]])
y = df["ipca"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())

# Teste: coeficiente do desemprego deve ser negativo
print(f"\nCoeficiente desemprego: {modelo.params['desemprego']:.4f}")
print(f"P-valor: {modelo.pvalues['desemprego']:.4f}")
```

5. Testes de Diagnóstico

5.1. Normalidade dos resíduos

```
from scipy import stats

residuos = modelo.resid

# Teste Jarque-Bera
jb_stat, jb_pval = stats.jarque_bera(residuos)
print(f"Jarque-Bera: {jb_stat:.4f} (p-valor: {jb_pval:.4f})")
# Se p > 0.05: resíduos normais (H0: normal)
```

```
# Q-Q plot
fig, ax = plt.subplots(figsize=(6, 6))
stats.probplot(residuos, dist="norm", plot=ax)
ax.set_title("Q-Q Plot dos Resíduos")
```

5.2. Heterocedasticidade (Breusch-Pagan)

```
from statsmodels.stats.diagnostic import het_breuschpagan

bp_test = het_breuschpagan(residuos, modelo.model.exog)
labels = ["LM Statistic", "LM p-valor", "F Statistic", "F p-valor"]
print(dict(zip(labels, bp_test)))
# Se p > 0.05: homocedasticidade (H0: homocedástico)
```

5.3. Autocorrelação (Durbin-Watson)

```
from statsmodels.stats.stattools import durbin_watson

dw = durbin_watson(residuos)
print(f"Durbin-Watson: {dw:.4f}")
# Próximo de 2: sem autocorrelação
# < 1.5: autocorrelação positiva
# > 2.5: autocorrelação negativa
```

6. Dados de Painei

```
# Dados simulados de painei
np.random.seed(42)
anos = range(2010, 2025)
países = ["Brasil", "Chile", "Colômbia", "Peru", "México"]

dados = []
for país in países:
    efeito_fixo = np.random.normal(0, 1)
    for ano in anos:
        pib = 2.0 + efeito_fixo + 0.5 * (ano - 2010) + np.random.normal(0, 1)
        inflacao = np.random.gamma(2, 2) + efeito_fixo
        dados.append({"país": país, "ano": ano, "pib": pib, "inflacao": inflacao})

df = pd.DataFrame(dados)

# Pooled OLS (ignora a estrutura de painei)
X = sm.add_constant(df[["inflacao"]])
y = df["pib"]
pooled = sm.OLS(y, X).fit()
print("POOLED OLS")
print(pooled.summary())

# Efeitos Fixos (dummies por país)
X_fe = sm.add_constant(pd.get_dummies(df[["inflacao", "país"]], columns=["país"],
drop_first=True))
y = df["pib"]
fe = sm.OLS(y, X_fe).fit()
print("\nEFEITOS FIXOS")
print(fe.summary())
```

Usando linearmodels (mais fácil)

```
from linearmodels.panel import PanelOLS, RandomEffects

df = df.set_index(["pais", "ano"])

# Efeitos Fixos
fe = PanelOLS.from_formula("pib ~ inflacao + EntityEffects", data=df)
print(fe.fit())

# Efeitos Aleatórios
re = RandomEffects.from_formula("pib ~ inflacao", data=df)
print(re.fit())

# Teste de Hausman (fixo vs aleatório)
from linearmodels.panel import compare
print(compare({"FE": fe.fit(), "RE": re.fit()}))
```

7. Séries Temporais

7.1. Estacionariedade (ADF Test)

```
ipca = sgs.get({"IPCA": 433}, start="2000-01-01")

# Teste ADF
result = adfuller(ipca["IPCA"].dropna())
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-valor: {result[1]:.4f}")
print(f"Valores críticos:")
for key, value in result[4].items():
    print(f"    {key}: {value:.4f}")

# Se p-valor < 0.05: série é estacionária
```

7.2. ARIMA

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# ACF e PACF para identificar ordem
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(ipca["IPCA"], lags=24, ax=ax1)
plot_pacf(ipca["IPCA"], lags=24, ax=ax2)
fig.tight_layout()

# ARIMA(1,0,1) – ajuste conforme ACF/PACF
modelo = ARIMA(ipca["IPCA"], order=(1, 0, 1))
resultado = modelo.fit()
print(resultado.summary())

# Previsão 12 meses
forecast = resultado.forecast(steps=12)
print("Previsão para os próximos 12 meses:")
print(forecast)

# Plotar previsão
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(ipca.index[-24:], ipca["IPCA"][-24:], label="Histórico")
```

```
ax.plot(forecast.index, forecast, label="Previsão", color="red", linestyle="--")
ax.legend()
ax.set_title("IPCA - Previsão ARIMA(1,0,1)")
fig.tight_layout()
```

7.3. Seleção automática de ARIMA

```
from statsmodels.tsa.arima.model import ARIMA
import itertools

# Grid search dos melhores p, d, q
p = range(0, 4)
d = range(0, 2)
q = range(0, 4)

best_aic = float("inf")
best_order = None

for pd_param, d_param, q_param in itertools.product(p, d, q):
    try:
        modelo = ARIMA(ipca["IPCA"], order=(pd_param, d_param, q_param))
        resultado = modelo.fit()
        if resultado.aic < best_aic:
            best_aic = resultado.aic
            best_order = (pd_param, d_param, q_param)
    except:
        continue

print(f"Melhor ARIMA{best_order} com AIC = {best_aic:.2f}")
```

8. Cointegração (Engle-Granger)

```
from statsmodels.tsa.stattools import coint

# Testar se duas variáveis andam juntas no longo prazo
cambio = sgs.get({"CAMBIOS": 1}, start="2000-01-01") # taxa de câmbio
selic = sgs.get({"SELIC": 11}, start="2000-01-01")

df = pd.merge(cambio, selic, on="Date").dropna()

score, pvalue, _ = coint(df.iloc[:, 0], df.iloc[:, 1])
print(f"Teste de Cointegração - p-valor: {pvalue:.4f}")
if pvalue < 0.05:
    print("As séries são cointegradas (andam juntas no longo prazo)")
else:
    print("Não há evidência de cointegração")
```

9. Modelo VAR

```
from statsmodels.tsa.api import VAR

# Variáveis: PIB, IPCA, SELIC, Câmbio
pib = sgs.get({"PIB": 4380}, start="2005-01-01") # PIB mensal
ipca = sgs.get({"IPCA": 433}, start="2005-01-01")
selic = sgs.get({"SELIC": 11}, start="2005-01-01")

df = pd.merge(pib, ipca, on="Date")
```

```

df = pd.merge(df, selic, on="Date").dropna()
df.columns = ["pib", "ipca", "selic"]

# Estimação
modelo = VAR(df)
resultado = modelo.fit(maxlags=6, ic="aic")
print(f"Defasagens ótimas: {resultado.k_ar}")
print(resultado.summary())

# Função impulso-resposta
irf = resultado.irf(12)
fig = irf.plot()
fig.tight_layout()

# Previsão
forecast = resultado.forecast(df.values[-resultado.k_ar:], steps=6)
forecast_df = pd.DataFrame(forecast, columns=df.columns)
print("Previsão 6 meses:")
print(forecast_df)

```

10. Teste de Causalidade de Granger

```

from statsmodels.tsa.stattools import grangercausalitytests

# SELIC causa IPCA?
gc = grangercausalitytests(df[["ipca", "selic"]], maxlag=6, verbose=False)

for lag in range(1, 7):
    p = gc[lag][0]["ssr_ftest"][1]
    print(f"Lag {lag}: p-valor = {p:.4f} {'Causa' if p < 0.05 else 'Não causa'}")

```

11. Exercícios

Exercício 1

Baixe dados de IPCA e SELIC de 2010 a 2026. Faça uma regressão de IPCA contra SELIC defasada em 1 mês. Interprete.

Exercício 2

Teste a Curva de Phillips para o Brasil: regreda IPCA contra desemprego e IPCA defasado.

Exercício 3

Encontre o melhor modelo ARIMA para o IPCA e faça previsão para 12 meses.

Exercício 4

Teste se SELIC causa IPCA no sentido de Granger. Em quais defasagens?

Exercício 5

Monte um painel com 5 países (dados anuais, 2010-2025) e teste a relação entre PIB e inflação.

12. Resumo — econometria

Modelo	Código
OLS	<code>sm.OLS(y, sm.add_constant(X)).fit()</code>
Regressão com fórmula	<code>smf.ols("y ~ x1 + x2", data=df).fit()</code>
Painel (fe)	<code>PanelOLS.from_formula("y ~ x + EntityEffects", data=df)</code>
ARIMA	<code>ARIMA(y, order=(p,d,q)).fit()</code>
ADF Test	<code>adfuller(serie)</code>
Cointegração	<code>coint(y1, y2)</code>
VAR	<code>VAR(df).fit(maxlags=6)</code>
Granger	<code>grangercausalitytests(df, maxlag=6)</code>